Multi-Prototype Neighborhood Distribution Learning for Node Classification

Leon Walcher
leon.walcher@uni-ulm.de
Ulm University
Ulm, Germany

Abstract

Graph Neural Networks (GNNs) show state-of-the-art performance in node classification. Recent work has addressed the issue that standard GNNs lag in performance on heterophilic graphs, while being a powerful node classification model on homophilic graphs. Specialized GNNs for heterophilic graphs aim to solve this problem, but recent work has shown that the inherent issue for GNNs is the differences in the neighborhood feature structure of the nodes to be classified, rather than heterophily in general. To overcome this problem, we introduce **MPNDL**, a model that combines GNNs with the concept of prototypical learning. MPNDL generates multiple prototype distributions of learned neighborhood embeddings per class. The node embeddings are created by a GNN encoder. We evaluate our method on six heterophilic and six homophilic datasets and compare it to five state-of-the-art baselines. Further, we give valuable insights into the mechanism behind MPNDL by analyzing the structure of the learned prototype distributions.

1 Introduction

Node classification is a fundamental task on graph-structured data, with many applications such as classifying entities in knowledge graphs [12, 27] or users in social graphs [1]. Successful node classification requires capturing both node features and neighborhood structure. Graph Neural Networks (GNNs) are state-of-the-art models for this task, typically learning node embeddings by aggregating neighborhood features. A key challenge is that embeddings of nodes belonging to the same class can differ substantially. In heterophilous graphs, neighbors often have different labels and attributes than the node itself, leading to dissimilar embeddings for same-class nodes. However, this issue also occurs in homophilous graphs. Prior work [9] shows that same-class embeddings can vary widely, particularly when nodes are far apart in the graph. Several approaches address this problem. Some GNN architectures are specifically designed for heterophilous graphs [4, 18, 21, 30]. Graph rewiring methods [9] modify edges to align neighborhoods of same-class nodes, but are computationally expensive and typically sample edges independently, ignoring full neighborhood distributions that are often crucial for classification. Graph transformers [28] consider the entire graph structure, but may lose local neighborhood information. Prototype-based methods learn class prototypes to handle intra-class embedding variability. However, existing approaches [5, 29] learn only embedding prototypes or capture neighborhood structure only implicitly. Following the paradigm of multi-prototype learning, we introduce **M**ulti-**P**rototype **N**eighborhood **D**istribution **L**earning (**MPNDL**). MPNDL explicitly models multiple neighborhood distribution prototypes per class and classifies nodes by comparing their neighborhood feature distributions to these prototypes. The prototypes are obtained by clustering neighborhood distributions of labeled training nodes within each class, and learning a representative distribution per cluster. The number of prototypes per class is adaptive and not a predefined hyperparameter. In summary, our contributions are:

- We introduce MPNDL, a node-classification method that learns multiple reference neighborhood distributions per class and uses them as class-specific prototypes.
- We evaluate MPNDL on twelve datasets, comparing its classification accuracy and computation time against strong baseline models.
- We conduct a comprehensive analysis of the number of learned prototypes for both heterophilic and homophilic benchmarks using comparable training-split sizes.

2 Related Work

We present special GNNs for heterophilic graphs, which we use in this paper. Further summarize existing work that uses prototypical learning for node classification. Additional related work on GNNs for heterophilic graphs is described in Appendix A.1.

GNNs for heterophilic Graphs. Zhu et al. [30] learn a compatibility matrix that models homophily or heterophily by capturing the likelihood of connections between nodes of different classes. Luan et. al. [18] propose an adaptive channel mixing strategy that learns how to combine aggregation, diversification, and identity channels based on the information that is important for each node.

Prototypical Learning for Node Classification. POWN [10] is a True-Open-World algorithm that combines semi-supervised and self-supervised learning via label propagation to learn prototype embeddings for known and unknown classes during training. ProtoGNN [5] processes the node features and the graph structure in two separate steps known as feature- and structure-view to enable long-range knowledge transfer of same-class nodes. In the feature view, multiple prototype embeddings are learned.

Zhang et al. [29] introduced a self-interpretable model for graph classification that learns multiple class-specific prototype embeddings and performs inference by comparing inputs to these learned prototypes. The method can also be applied for node classification by considering the local graph around the node of interest. This method is inherently different from our method, as (i) the prototypes are single vectors rather than distributions and learned differently, (ii) the number of prototypes is a fixed hyperparameter.

3 MPNDL

Graphs. The input of our model is a Graph $G = (V, E)$ represented by a set of nodes V and a set of edges $E \subseteq V \times V$. The set of neighbors of a node $v \in V$ is noted as $N(v)$. Each node has d features. The features for all nodes are represented by a feature matrix $\mathbf{X} \in \mathbb{R}^{|V| \times d}$. We operate in a **transductive** setup. Therefore, the graph structure (V, E) and all features $\mathbf{X}$ are visible during training. Also, the labels of the nodes V_L in the train split are visible, while all other labels are not.

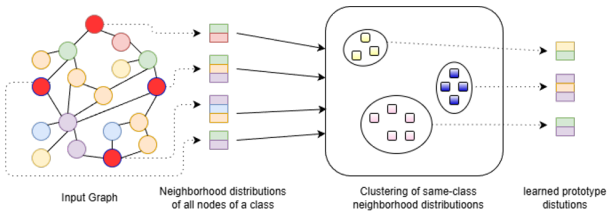

Figure 1: MPNDL prototype generation workflow for each class. First, the neighborhood feature distribution of each node of class c is extracted. Then the distributions are clustered, and one prototype is generated per cluster.

Our method. Figure 1 shows a high-level overview of how prototypes are generated using MPNDL. Our method first uses an arbitrary learnable encoder to generate a new feature representation $\mathbf{F} \in \mathbb{R}^{|V| \times d'}$ out of the feature matrix $\mathbf{X}$. The second step is performed per label class. The train nodes V_L are divided into C subsets $V_1, ..., V_C$ based on their label. Out of V_i a subset $V_i^{(T)}$ of nodes is sampled. The corresponding feature matrix is $\mathbf{F}_i^{(T)}$. For each node $u \in V_i^{(T)}$ the features of each direct neighbor are extracted and summarized in an unordered set $D_u = \{(\mathbf{F}_i^{(T)})_{v,:}\}_{v \in N(u)}$ representing the neighborhood feature distribution. Therefore, the cardinality of D_u is the degree of node u. This procedure leads to a set of distribution representatives $D^c = \{D_u | u \in V_i^{(T)}\}$ for each class c.

In a subsequent step, the $D_u \in D^c$ are assigned to k_c distinct clusters. A differentiable distance measure $d(D_i, D_j)$ is used to quantify the dissimilarity between neighborhood distributions. For clustering, any (non-learnable) clustering algorithm may be employed. The number of clusters k_c can vary across different classes and is determined by the clustering algorithm. For each cluster κ_l^c of class c, a prototype distribution $\mathcal{P}_l^i$ is defined, as the union of all samples in the clustered distribution, i.e. $\mathcal{P}_l^i = \bigcup_{D \in \kappa_l^c} D$. To learn the encoder the remaining train nodes $V^{(R)} = \bigcup_{i=1}^{C} (V_L \setminus V_i^{(T)})$ are used. For each node u of class c, the embeddings of the neighboring nodes are collected, so that the node is represented by $D_u = \{\mathbf{F}_{v,:}\}_{v \in N(u)}$. In a first step, the closest same-class prototype is determined, as

$$l^* = \arg\min_l d(D_u, \mathcal{P}_l^c) \tag{1}$$

We then update the encoder to minimize the distance between D_u and $P_{l^*}^c$. Therefore, we use the following loss function:

$$\mathcal{L}_{pos} = d(D_u, \mathcal{P}_{l^*}^c) \tag{2}$$

Regularization. $\mathcal{L}_{pos}$ yet only pushes D_u to the closest same-class prototype distribution. To ensure that D_u is not pushed in the direction of different-class prototypes, we introduce a regularization term

$$\mathcal{L}_{neg} = \sum_{c' \neq c} \sum_{l=1}^{k_{c'}} \max(0, m - d(D_u, P_l^{c'})) \tag{3}$$

with margin m, depending on the measure d. This regularization term pushes away D_u from different-class prototypes $\mathcal{P}_{l^*}^{c'}$.

Inference. For inferring the label of a node $u \in V$, the neighborhood feature distribution D_u is determined as presented above. Then D_u is compared to all learned prototype distributions, and the class of the closest prototype is chosen to be the label of u, i. e., $y = \arg\min_c (\min_l d(D_u, P_l^c))$.

Dataset	|V|	|E|	C	LI [1]
Cora	2,708	10,556	7	0.59
Citeseer	3,327	9,104	6	0.45
PubMed	19,717	88,648	3	0.41
FullCora	19,793	126,842	70	0.52
OGBn-ArXiv	169,343	1,166,243	40	0.45
OGBn-Products	2,449,029	123,718,280	47	0.64
Chameleon	2,277	36,101	5	0.05
Squirrel	5,201	217,073	5	0.00
Actor	7,600	30,019	5	0.00
Roman-Empire	22,662	65,854	18	0.11
Amazon-Ratings	24,492	186,100	5	0.04
Wisconsin	251	515	5	0.12

Table 1: Characteristics of the datasets. The table covers the number of nodes (|V|), edges (|E|), and classes (C), as well as the label informativeness score (LI).

Our setup. As backbone encoders, we use GCN, GraphSAGE, and ACM-GCN, leading to three different variants of MPNDL. For all MPNDL models, we use the squared maximum mean discrepancy [6] as a distance measure, i. e., $d(D_u, D_v) = \text{MMD}^2(D_u, D_v)$. For clustering, we use an optimized version [19, 20] of HDBSCAN [3] with MMD^2 as distance metric.

4 Experimental Apparatus

4.1 Datasets

We use a total of twelve datasets, of which six are homophilic and six are heterophilic. The homophilic datasets are the citation datasets Cora, Citeseer, and PubMed [26], as well as FullCora [2], OGBn-ArXiv, and OGBn-Products [11]. Furthermore, we use the heterophilic datasets Chameleon, Squirrel, and Actor [21]. Additionally, we use the heterophilic Roman-Empire, Amazon-Ratings [23], and Wisconsin [21] dataset.

4.2 Procedure

We evaluate our model against five strong baselines, GCN [15], GAT [25], GraphSAGE [8], ACM-GCN [18], and LINKX [16]. We train MPNDL on the train split of twelve real-world datasets and optimize the hyperparameters based on the validation split (see Section 4.3 for details). Furthermore, we perform a deep analysis of the learned prototype feature distributions.

Benchmark datasets. As the number of train nodes per class is crucial for the number of prototypes that MPNDL can learn, we ensure a comparable setup among all datasets. To do so, we follow Shchur et al. [24] and create ten random train/validation/test splits, with each train split containing 20 nodes per class and each validation split containing 30 nodes per class. The rest of the labeled nodes are assigned to the test splits.

Analysis of prototype distributions. We compare the number of learned prototype distributions per class. Further, we analyze the distances of the prototype distributions per class by comparing them to the distances between different-class prototype distributions.

[1] We calculated all LI scores using the DGL implementation. Only on OGBn-Products we find a deviation from the values reported in the introductory paper on LI score.

4.3 Hyperparameter Optimization

We use grid search to optimize the hyperparameters. For all baselines, we tune the number of layers, the number of units per hidden layer, the learning rate, and the dropout rate. For GAT, we further tune the number of attention heads. For MPNDL, we only tune the hyperparameters of the backbone encoder and the clustering algorithm. We use either GCN, GraphSAGE, or ACM-GCN as encoder. We optimize all models using the Adam optimizer [14]. The resulting search spaces and final hyperparameters per model are presented in Appendix A.2.

4.4 Measures

To measure the classification performance of a model, we use the accuracy averaged over ten runs.

To measure the similarity of two distributions P and Q based on samples $\{x_1, ..., x_m\} \sim P$ and $\{y_1, ..., y_n\} \sim Q$ we use the squared maximum mean discrepancy MMD^2 [6] defined as:

$$\text{MMD}^2 = \frac{1}{m^2}\sum_{i,j}^{m} k(x_i, x_j) - \frac{2}{mn}\sum_{i,j}^{m,n} k(x_i, y_j) + \frac{1}{n^2}\sum_{i,j}^{n} k(y_i, y_j)$$

with gaussian kernel $k(u, v) = \exp\left(-\frac{||u-v||^2}{\sigma^2}\right)$ and bandwidth σ^2.

To measure the similarity between the neighborhood structures of nodes within a class, we calculate the label informativeness [22] defined as:

$$\text{LI} = I(y_u, y_v)/H(y_u) \tag{4}$$

where y_u and y_v are the labels of two connected nodes $u, v \in E$. LI is the mutual information $I(y_u, y_v)$ between y_u and y_v normalized by the entropy $H(y_u)$ of y_u.

Generative AI Disclosure

Generative Artificial Intelligence (AI) was used to reformulate sentences. Further, we partially used AI-Tools for literature search and consistency checking. Generative AI was not used for generating figures, tables or entire text passages.

References

[1] Smriti Bhagat, Graham Cormode, and S. Muthukrishnan. 2011. Node Classification in Social Networks. *CoRR* abs/1101.3291 (2011). arXiv:1101.3291 http://arxiv.org/abs/1101.3291

[2] Aleksandar Bojchevski and Stephan Günnemann. 2018. Deep Gaussian Embedding of Graphs: Unsupervised Inductive Learning via Ranking. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. https://openreview.net/forum?id=r1ZdKJ-0W

[3] Ricardo J. G. B. Campello, Davoud Moulavi, and Joerg Sander. 2013. Density-Based Clustering Based on Hierarchical Density Estimates. In *Advances in Knowledge Discovery and Data Mining*, Jian Pei, Vincent S. Tseng, Longbing Cao, Hiroshi Motoda, and Guandong Xu (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 160–172.

[4] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2021. Adaptive Universal Generalized PageRank Graph Neural Network. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. https://openreview.net/forum?id=n6jl7fLxrP

[5] Yanfei Dong, Mohammed Haroon Dupty, Lambert Deng, Yong Liang Goh, and Wee Sun Lee. 2023. ProtoGNN: Prototype-Assisted Message Passing Framework for Non-Homophilous Graphs. https://openreview.net/forum?id=LeZ39Gkwbi0

[6] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. 2012. A Kernel Two-Sample Test. *Journal of Machine Learning Research* 13, 25 (2012), 723–773. http://jmlr.org/papers/v13/gretton12a.html

[7] Arthur Gretton, Bharath K. Sriperumbudur, Dino Sejdinovic, Heiko Strathmann, Sivaraman Balakrishnan, Massimiliano Pontil, and Kenji Fukumizu. 2012. Optimal kernel choice for large-scale two-sample tests. In *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger (Eds.). 1214–1222. https://proceedings.neurips.cc/paper/2012/hash/dbe272bab69f8e13f14b405e038deb64-Abstract.html

[8] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7ebea9-Paper.pdf

[9] Marcel Hoffmann, Lukas Galke, and Ansgar Scherp. 2025. Gumbel-MPNN: Graph Rewiring with Gumbel-Softmax. *CoRR* abs/2508.17531 (2025). https://doi.org/10.48550/ARXIV.2508.17531 arXiv:2508.17531

[10] Marcel Hoffmann, Lukas Galke, and Ansgar Scherp. 2025. POWN: Prototypical Open-World Node Classification. In *Proceedings of The 3rd Conference on Lifelong Learning Agents (Proceedings of Machine Learning Research, Vol. 274)*, Vincenzo Lomonaco, Stefano Melacci, Tinne Tuytelaars, Sarath Chandar, and Razvan Pascanu (Eds.). PMLR, 672–691. https://proceedings.mlr.press/v274/hoffmann25a.html

[11] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). https://proceedings.neurips.cc/paper/2020/hash/fb60d411a5c5b72b2e7d3527cfc84fd0-Abstract.html

[12] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. 2022. A Survey on Knowledge Graphs: Representation, Acquisition, and Applications. *IEEE Trans. Neural Networks Learn. Syst.* 33, 2 (2022), 494–514. https://doi.org/10.1109/TNNLS.2021.3070843

[13] Shixiong Jing, Lingwei Chen, Quan Li, and Dinghao Wu. 2024. H 2 GNN: Graph Neural Networks with Homophilic and Heterophilic Feature Aggregations. In *International Conference on Database Systems for Advanced Applications*. Springer, 342–352.

[14] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). http://arxiv.org/abs/1412.6980

[15] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net. https://openreview.net/forum?id=SJU4ayYgl

[16] Derek Lim, Felix Hohne, Xiuyu Li, Sijia Linda Huang, Vaishnavi Gupta, Omkar Bhalerao, and Ser-Nam Lim. 2021. Large Scale Learning on Non-Homophilous Graphs: New Benchmarks and Strong Simple Methods. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (Eds.). 20887–20902. https://proceedings.neurips.cc/paper/2021/hash/ae816a80e4c1c56caa2eb4e1819cbb2f-Abstract.html

[17] Mingsheng Long, Yue Cao, Jianmin Wang, and Michael I. Jordan. 2015. Learning Transferable Features with Deep Adaptation Networks. In *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015 (JMLR Workshop and Conference Proceedings, Vol. 37)*, Francis R. Bach and David M. Blei (Eds.). JMLR.org, 97–105. http://proceedings.mlr.press/v37/long15.html

[18] Sitao Luan, Chenqing Hua, Qincheng Lu, Jiaqi Zhu, Mingde Zhao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. 2022. Revisiting Heterophily For Graph Neural Networks. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/092359ce5cf60a80e882378944bf1be4-Abstract-Conference.html

[19] Leland McInnes and John Healy. 2017. Accelerated Hierarchical Density Based Clustering. In *2017 IEEE International Conference on Data Mining Workshops (ICDMW)*. 33–42. https://doi.org/10.1109/ICDMW.2017.12

[20] Leland McInnes, John Healy, and Steve Astels. 2017. hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software* 2, 11 (2017), 205.

[21] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-GCN: Geometric Graph Convolutional Networks. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net. https://openreview.net/forum?id=S1e2agrFvS

[22] Oleg Platonov, Denis Kuznedelev, Artem Babenko, and Liudmila Prokhorenkova. 2023. Characterizing Graph Datasets for Node Classification: Homophily-Heterophily Dichotomy and Beyond. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 523–548. https://proceedings.neurips.cc/paper_files/paper/2023/file/01b681025fdbda8e935a66cc5bb6e9de-Paper-Conference.pdf

[23] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. 2023. A critical look at the evaluation of GNNs under

heterophily: Are we really making progress?. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=tJbbQfw-5wv

[24] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Pitfalls of Graph Neural Network Evaluation. arXiv:1811.05868 [cs.LG] https://arxiv.org/abs/1811.05868

[25] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. https://openreview.net/forum?id=rJXMpikCZ

[26] Zhilin Yang, William W. Cohen, and Ruslan Salakhutdinov. 2016. Revisiting Semi-Supervised Learning with Graph Embeddings. In *Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016 (JMLR Workshop and Conference Proceedings, Vol. 48)*, Maria-Florina Balcan and Kilian Q. Weinberger (Eds.). JMLR.org, 40–48. http://proceedings.mlr.press/v48/yanga16.html

[27] Zi Ye, Yogan Jaya Kumar, Goh Ong Sing, Fengyan Song, and Junsong Wang. 2022. A Comprehensive Survey of Graph Neural Networks for Knowledge Graphs. *IEEE Access* 10 (2022), 75729–75741. https://doi.org/10.1109/ACCESS.2022.3191784

[28] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do Transformers Really Perform Badly for Graph Representation?. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 28877–28888. https://proceedings.neurips.cc/paper_files/paper/2021/file/f1c1592588411002af340cbaedd6fc33-Paper.pdf

[29] Zaixi Zhang, Qi Liu, Hao Wang, Chengqiang Lu, and Cheekong Lee. 2022. ProtGNN: Towards Self-Explaining Graph Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 8 (Jun. 2022), 9127–9135. https://doi.org/10.1609/aaai.v36i8.20898

[30] Jiong Zhu, Ryan A. Rossi, Anup Rao, Tung Mai, Nedim Lipka, Nesreen K. Ahmed, and Danai Koutra. 2021. Graph Neural Networks with Heterophily. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*. AAAI Press, 11168–11176. https://doi.org/10.1609/AAAI.V35I12.17332

[31] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). https://proceedings.neurips.cc/paper/2020/hash/58ae23d878a47004366189884c2f8440-Abstract.html

A Supplementary Materials

A.1 Extended Related Work

Here we summarize additional related work in the area of heterophilic GNNs. GeomGCN [21] maps the graph to a latent space, with another graph structure, and then uses a bi-level aggregator operating on both graphs to create node feature representations. GPR-GNN [4] combines GNNs with PageRank techniques by learning PageRank-based propagation weights, enabling the aggregator to adapt to different label structures in a graph. H_2GCN [31] combines the principles of separation between ego and neighborhood embeddings, high-order neighborhood aggregation, and combination of intermediate representations. Jing. et al. [13] introduced a method that learns two local feature vectors for each node, distinguishing between similar and dissimilar features. The feature propagation is done adaptively depending on whether a connection is identified as homophilic or heterophilic. LINKX [16] separates the graph structure and node features by encoding node features and the adjacency matrix by MLPs. After that, the concatenation of both embeddings is fed through yet another MLP.

A.2 Hyperparameter Optimization

We tune all hyperparameters by grid search over all parameter combinations presented below. For all models, we optimize the learning rate of the Adam [14] optimizer in $\{0.001, 0.01, 0.1\}$.

Baselines. For all baselines, namely, GCN, GAT, GraphSAGE, ACM-GCN, and LINKX we optimize the number of units per hidden layer in $\{128, 256, 512, 1024\}$, the number of hidden layers in $\{1, 2, 3\}$, and the dropout rate in $\{0, 0.2, 0.5\}$. For GAT, we additionally tune the number of attention heads in $\{2, 4, 8\}$.

Our Model. We construct MPNDL using three backbone encoder GNNs, namely GCN, GraphSAGE, and ACM-GCN, resulting in three model variants: GCN-MPNDL, GS-MPNDL, and ACM-MPNDL. For all the backbone models, we use the same search space as for the baseline models. Additionally, for MPNDL, we optimize the minimum cluster size of HDBSCAN in $\{3, 5, 10\}$. We set the bandwidth hyperparameter σ^2 of the Gaussian kernel to the median of the pairwise distances on the training samples, i. e., $\sigma^2 = \text{median}_{i,j}\{||x_i - x_j||_2^2\}$. This is widely known as the median heuristic [7, 17].